LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CC5009NI Cyber Security in Computing

Assessment Weightage & Type

40% Individual Coursework 01

Year and Semester

2025 -26 Autumn Semester

Student Name: Badal Kumar Yadav

London Met ID: 24046365

College ID: NP01NT4A240195

Assignment Due Date: January 19TH, 2025

Assignment Submission Date: January 19TH, 2025

Word Count (Where Required): 3499

I confirm that I understand my coursework needs to be submitted online via MST under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

TABLE OF CONTENT

TABLE OF FIGURES

TABLE OF TABLES

ABSTRACT

This coursework analyses variants of the conventional Feistel cipher algorithm that is elegant and invertible but tends to exhibit poor diffusion properties as well as poor avalanche characteristics in its simpler variants. The project proposes the design and implementation of the Avalanche Cloud Feistel Cipher (ACFC), which is a learning block cipher that uses 32-bit blocks, possesses a 64-bit key, operates through 8 rounds of processing, makes use of both modular addition and bit rotation of 1 bit, along with key whitening. The project encompasses five stages in its implementation: examining the basics of cryptography, choosing the Feistel layout approach, designing ACFC using simple and traceable processes, testing the process for different parameters such as short texts, empty input, long texts, similar patterns, and avalanche test cases, and analysing its weaknesses and strengths. This project has proved successful in encryption and decryption processes, avalanche effects, and its effectiveness in the classroom for learning purposes. ACFC should not be used in practical applications because of its small scale and simple key management mechanism; however, it can work wonders in the classroom for learning confusion, diffusion, and symmetry in the Feistel model.

1.0 Introduction

1.1 Security in Computing

Security in computing means protecting computers, networks, and data from wrong person and only right person access them, and nothing get damaged, stolen or changed without permission. It makes sure that information is confidential, accurate and available so only authorized person can see, change and can access the information. Many companies and industries like banks, hospitals and government services depends upon digital system which keeps them safe from attacker, so it is important in today world (hackerone, 2025).

1.2 The CIA Triad in Information Security

Information security is based on the CIA triad of confidentiality, integrity, and availability. Encryption and access controls are frequently used to enforce confidentiality, which guarantees that sensitive data is only accessible by authorized people or systems. Integrity, which is frequently confirmed using cryptographic hash functions or digital signatures, ensures that data is accurate, consistent, and unchanged during storage, transmission, or processing. Through redundancy, fault tolerance, and defense against denial-of-service attacks, availability guarantees that data and resources are available to authorized users when needed. In order to create effective security postures in contemporary computing environments, these three interdependent principles must be balanced (FORTINET, 2025).

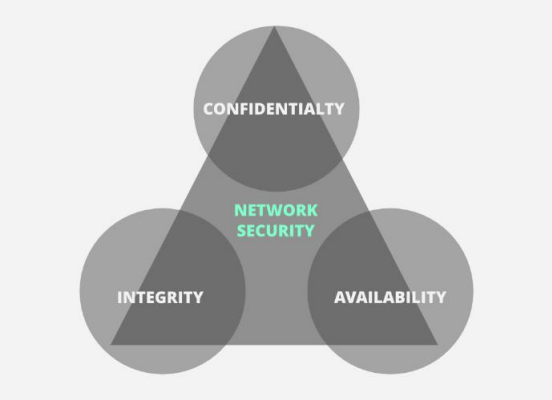


Figure 1: CIA Triad Diagram (GREEKS OF GREEKS, 2025)

1.3 Introduction to Cryptography

Cryptography can be defined as a field of science that uses mathematics to protect information with regard to confidentiality, integrity, and authentication. It is used to convert readable data into unreadable data (Adomey, 2026).

- Translates plaintext messages into ciphertext using encryption keys
- It provides confidentiality, integrity, authentication, and non-repudiation.
- Used for secure communication, digital signatures, password protection, and electronic transaction.

1.4 Key Cryptographic Terminologies

The key terminologies are given below:

- Plaintext/Ciphertext: Plaintext is the original, readable data; ciphertext is the encrypted output (greeksforgeeks, 2025).
- Encryption/Decryption: Encryption converts plaintext to ciphertext; decryption reverses the process.
- Key: A secret value used with an algorithm to encrypt or decrypt data. Symmetric keys are identical for both operations; asymmetric keys differ (public/private).
- Algorithm/Cipher: A set of mathematical rules governing encryption and decryption (greeksforgeeks, 2025).
- Cryptanalysis: The study of methods to break cryptographic systems.
- Hash Function: A one-way function producing a fixed-size output from input data, used for integrity verification.
- Digital Signature: A cryptographic technique verifying authenticity and integrity, based on asymmetric cryptography (greeksforgeeks, 2025).

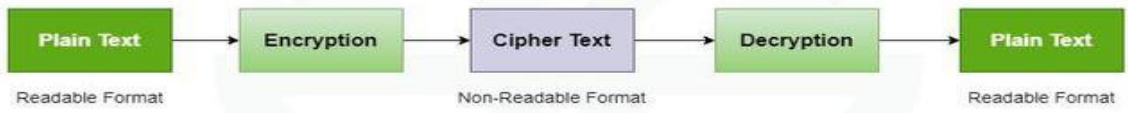


Figure 2: Cryptography and its Types (greeksforgeeks, 2025)

1.5 Aims and Objectives of the Coursework

1.1.1 Aim

The main aim of this coursework is to investigate, design, develop, and critically evaluate a new cryptographic algorithm using the Feistel cipher, thus proving the understanding of the concepts of cryptography in the area of cyber security.

1.1.2 Objectives

To achieve the stated aim, the following specific objectives have been defined:

- For analysing basic cryptographic ideas, such as security fundamentals, encryption methods, CIA triad, and the history of cryptography.
- For analysing the Feistel cipher based on its characteristics and features, and to see its applications.
- To develop an improved cryptographic algorithm based on the Feistel network.
- To execute and analyse the proposed algorithm to determine the accuracy, speed, and vulnerability to attacks.
- in order to determine possible real applications of the algorithm and evaluate the merits and demerits of the new algorithm.
- To record the research and development activities and results in an academic format with references.

2.0 Background of Cryptographic Algorithm: Feistel Cipher

2.1 Overview of the Feistel Cipher

The Feistel cipher, or Feistel network, is a symmetric block cipher method developed by Horst Feistel at IBM during the early 1970s. It was first used in the Lucifer cipher and later became the basis for the Data Encryption Standard, arguably the most important encryption method in history (NIST , 1999). The Feistel cipher is characterized by its well-balanced and iterative approach, where plaintext messages are divided into two equal halves to be processed through a number of transformations via a round function and round keys. One of the important characteristics of Feistel networks is that they are symmetric, so that a Feistel cipher circuit could be used for both encryption and decryption by simply reversing the order of the round keys this was an important innovation that made their implementation easier both in hardware and software. The Feistel cipher has been a model and inspiration for many other ciphers, such as Blowfish, Twofish, and Camellia, to name just a few, and remains an important cryptographic construct to this day (NIST , 1999) (Menezes, 1997).

2.2 Historical Development and Design Principles

The idea for the Feistel cipher was developed at Thomas J. Watson Research Center at IBM by Horst Feistel in the early 1970's because of an ever-growing need for a standardized commercial encryption method. The first prototype of the encryption method was named Lucifer and was put forward at the National Bureau of Standards (NBS), which is now the National Institute of Standards and Technology (NIST), in response to an announcement for a federal encryption method put forward by the NBS. The principles for Lucifer were later evolved to form the Data Encryption Standard (DES), which was released in 1977 as 'FIPS PUB 46' (NIST , 1999). The principles for the method are based on those defined by 'confusion' and 'diffusion' concepts put forward by Claude E. Shannon, where 'confusion' is implemented through substitution (usually through the use of 'S-boxes'), while 'diffusion' is implemented through permutation and bit manipulation through a number of rounds in the 'Feistel network' concept (NIST , 1999).

The idea for the Feistel network also incorporated another crucial aspect for the encryption method's success: an 'algorithm for key scheduling,' which generated a

'unique key for each round' from a 'master key,' so that each round presented 'distinct cryptographic' quality (Menezes, 1997).

2.3 Structure and Operation

The Feistel cipher operates on fixed-length blocks of plaintext, typically split into two halves: L_0 (left) and R_0 (right). Each round i performs the following transformations using a round function F and a round key K_i (mrajacse.wordpress, 2019):

$$L_i = R_{i-1}$$
$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

where $\oplus$ denotes the XOR operation. After a predetermined number of rounds n, the halves are concatenated (sometimes with a final swap) to produce the ciphertext. The **round function** F is crucial to security and may incorporate substitution tables (S-boxes), permutations, and arithmetic operations. Decryption uses the same structure but applies round keys in reverse order $(K_n, K_{n-1}, \ldots, K_1)$, leveraging the self-inverting property of each round. A typical Feistel round can be visualized as (mrajacse.wordpress, 2019):

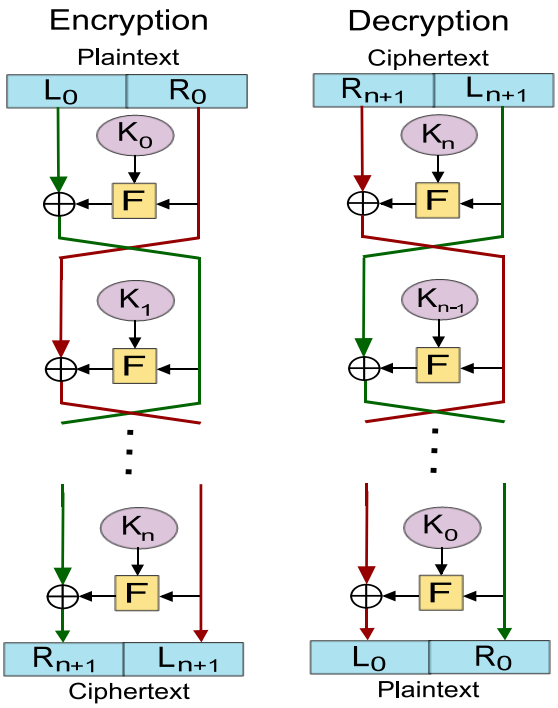


Figure 3: Feistel Cipher Process

This diagram illustrates the flow of data, highlighting the XOR operation and the swapping of halves.

2.4 Example: Step-by-Step Encryption

Let's take a simple 2-round 8-bit Feistel cipher with 4-bit blocks, round function $F(R,K) = R \oplus K$, and round keys $K_1 = 0101$, $K_2 = 1010$. Encrypt the plaintext 11000110:

1. **Initial split**:
   $L_0 = 1100$, $R_0 = 0110$
2. **Round 1**:
   $$F(R_0, K_1) = 0110 \oplus 0101 = 0011$$
   $$L_1 = R_0 = 0110$$
   $$R_1 = L_0 \oplus F = 1100 \oplus 0011 = 1111$$
3. **Round 2**:
   $$F(R_1, K_2) = 1111 \oplus 1010 = 0101$$
   $$L_2 = R_1 = 1111$$
   $$R_2 = L_1 \oplus F = 0110 \oplus 0101 = 0011$$
4. **Final output (with swap)**:
   Ciphertext = $R_2 \parallel L_2 = 00111111$

A decryption process reverses the steps described above by utilizing the round keys in the reverse sequence and effectively retrieving the message (mrajacse.wordpress, 2019).

2.5 Advantages of the Feistel Cipher

The advantages are given below (Bing sun, 2021):

- Structural Symmetry: The same algorithm for encryption and decryption.
- Flexibility of Design: The round function F is designed to allow customization and this doesn't affect the invertibility property as
- Proven Security - Foundation: Follows Shannon's principles of confusion and diffusion.
- Extensive Analysis: Detailed analysis is carried out, and the results are standardized

- Implementation Efficiency: Hardware & software can be fully efficiently implemented.
- Scalability: More round counts may improve the level of security.

2.6 Disadvantages of the Feistel Cipher

- Dependent on Round Function: The overall security of the entire cipher is reliant upon the round
- Performance Overhead: Many rounds in a protocol may
- Vulnerability to Side Channel Attacks: Predictable structure can leak information.
- Fixed Block Size Weaknesses: Requires padding, that could be a weakness.
- Obsolescence of Early Implementations: DES and other classical ciphers are susceptible to current cryptanalysis.
- Limited Parallelization: Sequential round dependency resists parallel computation.

3.0 Development of a New Cryptographic Algorithm

3.1 Research on Modifications to Make Feistel Cipher

The traditional Feistel cipher layout is one of the most beautiful examples of perfect symmetry in the design of symmetric key cryptography. However, the most trivial variants can often suffer from poor diffusion both in avalanche effect and mixing characteristics. The present pedagogical needs in cryptography instruction include implementations that produce:

- Very transparent and traceable in every round
- Able to demonstrate the avalanche criterion clearly through bit spreading
- Improve decipherments by incorporating essential modern methods such as modular arithmetic and key mixing
- Maintain entire symmetry in the algorithm for encryption as well as decryption.

The Avalanche Cloud Feistel Cipher (ACFC) has been developed in ways that meet these educational requirements and which maintain the key benefits of Feistel schemes. The ACFC designation emphasizes the diffusion characteristics afforded by the avalanche/cloud diffusion that the scheme provides, which is exceptionally useful for educational instruction related to confusions and diffusion.

3.2 Background of Newly Introduced Mathematical and Logical Operations

The Avalanche Cloud Feistel Cipher (ACFC) combines a very small but educationally powerful collection of operations:

I. **Exclusive-OR (XOR) Operation:** This is the fundamental mixing function between data halves and round keys. Reversible, commutative, as well as associative properties, make it particularly well-suited for Feistel-style structures at places where $A \oplus B \oplus B = A$ (Floyd, 2015).

II. **Modular Addition:** This is defined as $(A + B) \bmod 2^{16}$. This introduces carry propagation across bit positions, introducing non-linearity into the mixing

process, enhancing diffusion much more than a pure XOR-based mixing (GREEKSOFGREEKS, 2025).

III. **Left Circular Bit Rotation by 1 Position (ROTL1):** A lightweight diffusion primitive that performs a circular shift on bits, ensuring influence from any single bit position ripples through the entire 16-bit word during successive rounds (Nagarajan, 2017).

IV. **Key Whitening:** This includes taking the entire 32-bit block and XORing it with two 32-bit key-derived values-one prior to the first round, pre-whitening, and one after the final round, post-whitening. This step enhances general key diffusion and provides additional resistance against attacks that are based on the first or the last transformation steps (Bhatia & Som, 2016).

These four operations are specifically chosen for their simplicity in computation, clarity of concept, as well as their excellent capabilities in the explanation of cryptographic concepts.

3.3 Creation of New Encryption and Decryption Algorithm

A purposely simplified learning block cipher called the Avalanche Cloud Feistel Cipher (ACFC) has been developed by upgrading the traditional Feistel scheme with appropriate lightweight elements. The proposed design consists of a 32-bit block size, a 64-bit master key, and parameters for exactly eight rounds that are small enough for tracing by hand yet large enough for the diffusion phase to work effectively.

Encryption and decryption processes perform the same algorithmic steps. The difference only lies in the order of the round keys, which is decrypted in reverse order, a characteristic property of all Feistel networks.

3.4 Name of the New Cryptographic Algorithm

The newly created educational cipher is named:

Avalanche Cloud Feistel Cipher (ACFC)

This designation intentionally emphasizes:

- dramatic avalanche-like behavior in response to small inputs or crucial changes
- Cloud like diffusion and spreading of bit influence in the block and as well as mention from the name.
- clear connection to the basic Feistel network architecture.

The ACFC is produced solely for educational purposes as a teaching tool, not for any type of production security function.

3.5 Elaboration of the Avalanche Cloud Feistel Cipher

3.5.1 Why the Modification Was Necessary

Although the original Feistel network is mathematically correct and symmetric, too simple versions of it have failed to provide diffusion or avalanche properties adequately. If simply bitwise XOR is involved or a few rounds of it are performed, normally just a few bits of the output will change for just a single change in the input bit, making it of little use for illustration purposes.

The current state of cryptography education is helped along by:

- Permits full visibility of intermediate states
- incorporate basic non-linear and diffusion processes
- explain the cumulative impact of several rotations
- discuss practical key-mixing methods like whitening

The modifications introduced within ACFC remedy all these instructional flaws while maintaining as much simplicity and traceability as possible.

3.5.2 The New Methodology Implied

The Avalanche Cloud Feistel Cipher makes the following targeted education improvements:

a. **Compact Block Size:** 32 bits, split into two 16-bit blocks; allows for full representation of values in hexadecimal form.

b. **Modest Key Size:** 64 bits; enough for eight round keys and two whitening values via simple bit slicing.

c. **Balanced Round Count:** Eight rounds; ensures a progressive blending with enough complexity.

d. **Elementary Key Schedule:** Direct extraction of 16-bit round keys from consecutive non-overlapping segments of the master key.

e. **Symmetric Key Whitening:** Utilization of 32-bit XOR values calculated based on upper and lower parts of master keys prior and post Feistel processes.

f. **Lightweight Round Function:** Modular addition combined with a hardcoded 1-bit left rotation, optimal for spreading and non-linearity with minimal calculations.

g. **Avalanche Orientation:** Designed to propagate small changes in a visible manner throughout the block after a few rounds.

3.5.3 The New Encryption Algorithm

The encryption process of the Avalanche Cloud Feistel Cipher (ACFC) involves the systematic processing of 32-bit plaintext blocks with a 64-bit master key over eight rounds.

Algorithm Parameters

- Plaintext Block (P): 32 bits
- Master Key (K): 64 bits
- Number of Rounds (r): 8
- Block Halves: L_i and R_i, each 16 bits

Key Schedule Generation

From the 64-bit master key the following values are derived:

- Eight 16-bit round keys K_1 to K_8 through sequential 16-bit slicing
- Pre-whitening value W_pre = upper 32 bits of the master key
- Post-whitening value W_post = lower 32 bits of the master key

Round Function Definition

$F(R, K_i) = ROTL1_1((R + K_i) \bmod 2^{16})$

Where:

- + denotes integer addition
- mod 2^{16} restricts the result to 16 bits
- $ROTL1_1$ indicates left circular shift by one position

Encryption Procedure

Step 1: Initialization

Divide the 32-bit plaintext into:

- L_0 = most significant 16 bits
- R_0 = least significant 16 bits

Step 2: Pre-Whitening

= Apply XOR mixing with the pre-whitening value to the entire block

Step 3: Feistel Rounds (i = 1 to 8)

For each round:

- Compute temp = (R_{i-1} + K_i) mod 2^{16}
- Compute F = $ROTL1_1$(temp)
- Set L_i = R_{i-1}
- Set R_i = L_{i-1} $\oplus$ F

Step 4: Post-Whitening

After completion of all rounds, apply XOR with the post-whitening value to the combined block

Step 5: Ciphertext Formation

Concatenate the final left and right halves to produce the 32-bit ciphertext

3.5.4 The New Decryption Algorithm

The decryption algorithm exploits the natural symmetry of the Feistel construction and performs the same series of operations, but using the round keys in reverse order.

Decryption Process:

Step 1: Preparation

Split the 32-bit ciphertext into initial left and right halves

Step 2: Reverse post-whitening

Apply XOR with the post-whitening value to recover the state before final whitening

Step 3: Reverse Feistel Rounds (i = 8 down to 1)

For each round in reverse order:

- Compute the round function F using the corresponding round key
- Update halves using the standard Feistel swap-and-XOR pattern

Step 4: Reverse Pre-Whitening

After completing all reverse rounds, apply XOR with the pre-whitening value

Step 5: Plaintext Recovery

Recombine the recovered halves to obtain the original 32-bit plaintext

Algorithmic Symmetry Verification

The correctness of the decryption process relies on the fundamental Feistel reversibility property:

Encryption round: $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$ $L_i = R_{i-1}$

Decryption round: $L_{i-1} = R_i \oplus F(L_i, K_i)$ $R_{i-1} = L_i$

This equivalence ensures perfect recovery when round keys are applied in opposite sequence.

Key Design Features

- Preservation of Feistel symmetry in both encryption/decryption
- High educational transparency owing to small state size
- Introduction of non-linearity through modular addition
- Effective diffusion by rotation and several rounds
- Greater key involvement through whitening stages
- Avalanche simulation with animation and photos

The Avalanche Cloud Feistel Cipher (ACFC) is a well-balanced learning cryptographic scheme that aptly conveys the basics of modern cryptography in an efficient and easy-to-understand manner. The Avalanche Cloud Feistel Cipher functions as a simple block cipher.

3.6.5 encryption and decryption process example:

- **Encryption Process Example:** click here to see example
- **Decryption Process Example:** Click here to see example

3.6 Python code of ACFC cipher

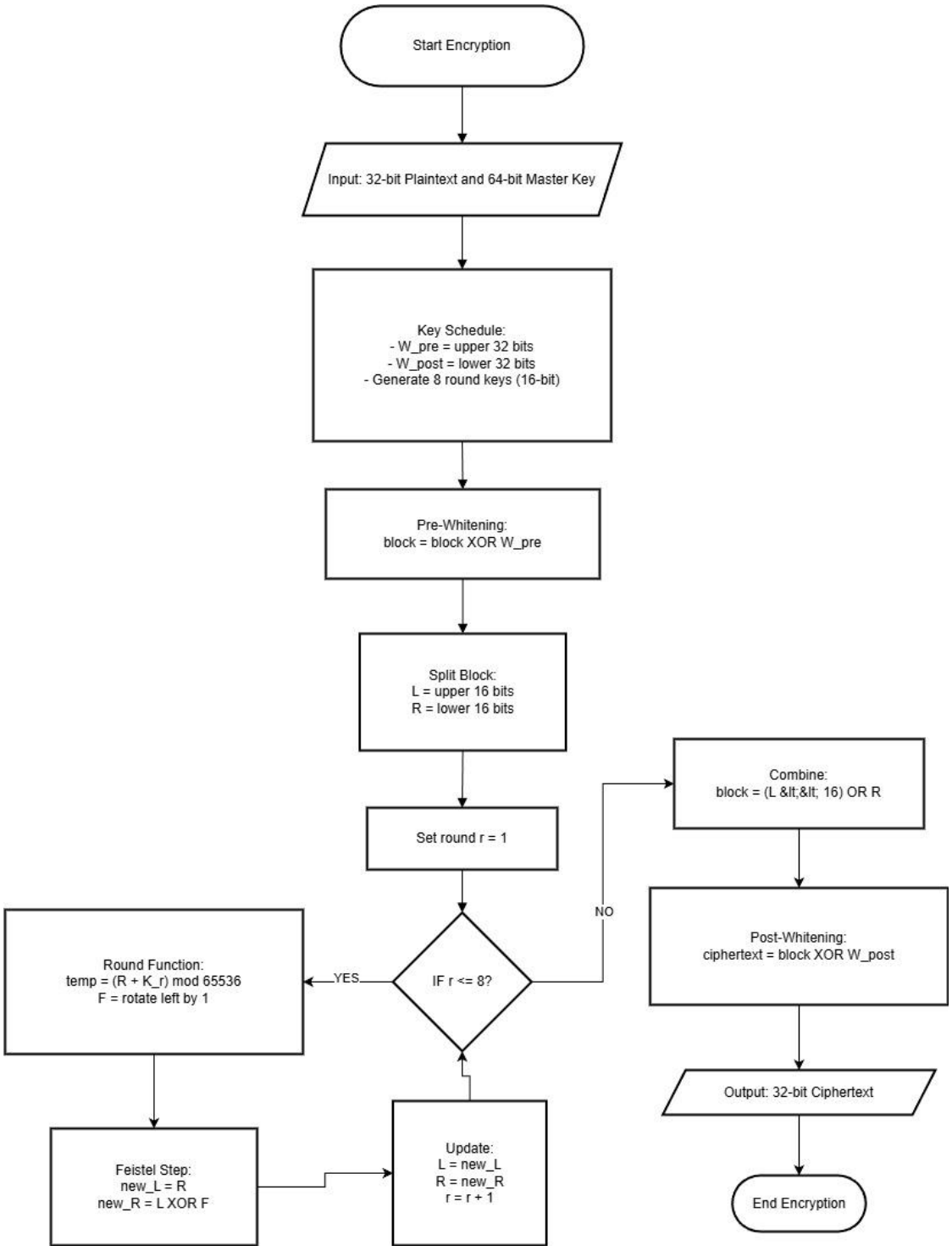
We need Python code because:
Manual encryption (like doing 8 rounds by hand) is slow, long, and error-prone.
Python can perform the same steps in milliseconds, exactly as in the flowchart.
It allows us to:

- Test different plaintexts and keys easily
- Verify that encryption and decryption work correctly
- Find bugs in logic (rounds, whitening, rotations)
- Show practical implementation, not just theory

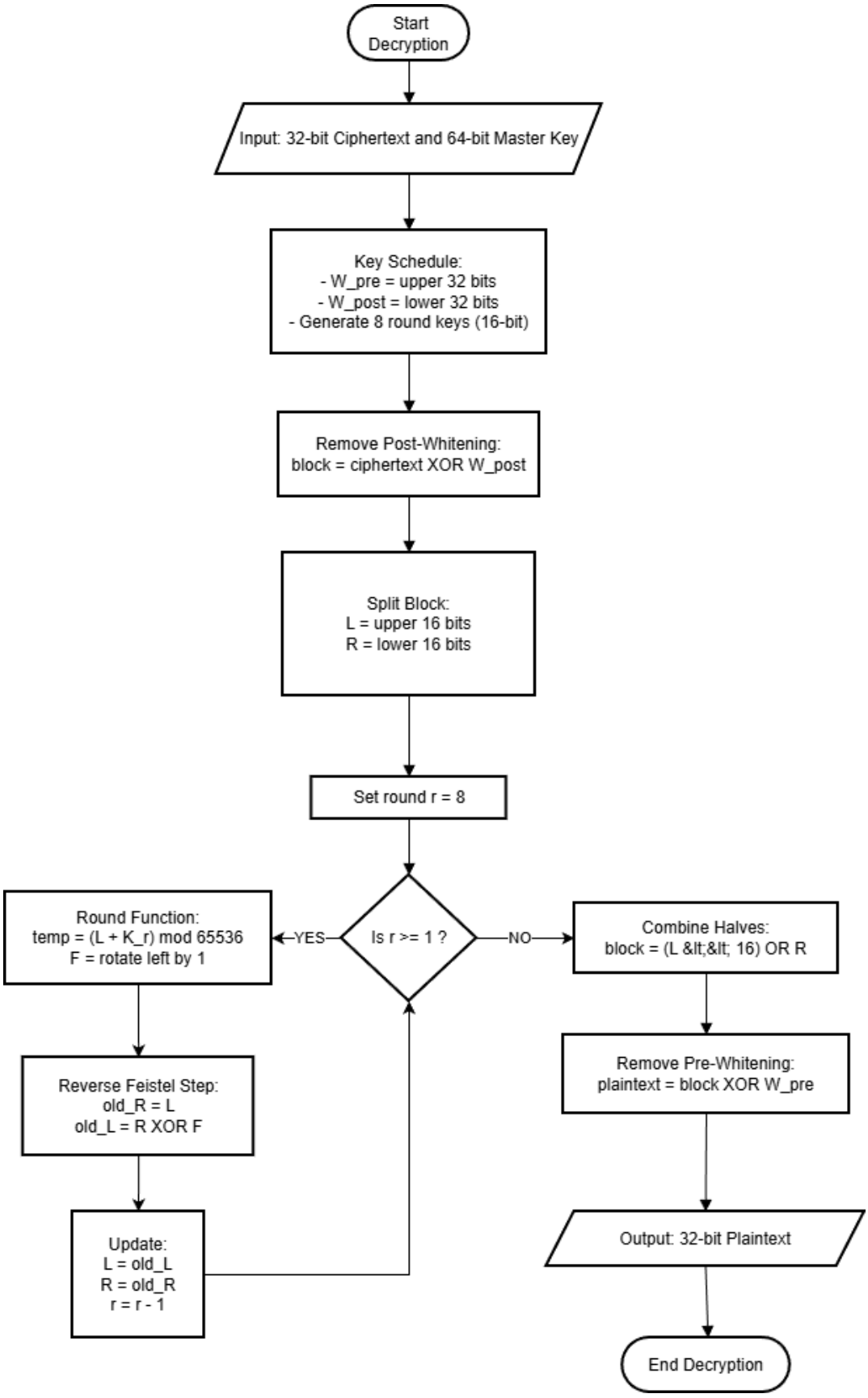
Click here to see code

3.6 Flowchart OF ACFC

3.6.1 Flowchart of Encryption Process



3.6.2 Flowchart of Decryption Process



4.0 TESTING

4.1 TEST 1: Basic round-trip correctness (short meaningful text)

- This test verifies that ensured that the encrypted form of a small text message can be decrypted into the same text message using the same key.
- **Plaintext:** Hello
- **Key:** 0123456789ABCDEF

Table 1: Table of Test 1

Test Objectives	**Verify basic encryption and decryption for short text.**
Test Input	➢ Plaintext: "HELLO" ➢ Key: 0123456789ABCDEF
Action Performed	Encrypt the plaintext and obtain ciphertext. Then decrypt with same key.
Expected Result	Encryption process plaintext and decryption process plaintext should be equal after using same master key.
Actual Result	➢ Ciphertext: 5DAEC203DF48D78F ➢ Recovered plaintext: "HELLO"
Conclusion	The Test was Successful. Encryption/Decryption process are working as expected.

```
=== Avalanche Cloud Feistel Cipher (ACFC)  ===

Key: 16 hex digits (64 bits)

 CHOOSE E- ENCRYPTION AND D- DECRYPTION: E
Key (16 hex digits): 0123456789ABCDEF
Plaintext: HELLO

------------------------------------------------------------------

Plaintext: 'HELLO'
Padded hex: 48454C4C4F030303
Blocks: 2

→ Plain block trace 0x48454C4C (Encrypt)
After initial whitening: 0x4966092B
     Initial: L=0x4966  R=0x092B
Round  1:    L=0x092B  R=0xE753   (k=0xCDEF  add=0xD71A  F=0xAE35)
Round  2:    L=0xE753  R=0xE8D7   (k=0x89AB  add=0x70FE  F=0xE1FC)
Round  3:    L=0xE8D7  R=0xBB2F   (k=0x4567  add=0x2E3E  F=0x5C7C)
Round  4:    L=0xBB2F  R=0x9072   (k=0x0123  add=0xBC52  F=0x78A5)
Round  5:    L=0x9072  R=0x9BCA   (k=0x0000  add=0x9072  F=0x20E5)
Round  6:    L=0x9BCA  R=0xA7E7   (k=0x0000  add=0x9BCA  F=0x3795)
Round  7:    L=0xA7E7  R=0xD405   (k=0x0000  add=0xA7E7  F=0x4FCF)
Round  8:    L=0xD405  R=0x0FEC   (k=0x0000  add=0xD405  F=0xA80B)
After final whitening: 0x5DAEC203

Ciphertext (hex):
5DAEC203DF48D78F
------------------------------------------------------------------
```

Figure 4: Encryption Process of Test 1

```
Continue? (y/n): Y
 CHOOSE E- ENCRYPTION AND D- DECRYPTION: D
Key (16 hex digits): 0123456789ABCDEF
Ciphertext (hex): 5DAEC203DF48D78F

------------------------------------------------------------------

→ Cipher block trace 0x5DAEC203 (Decrypt)
After initial whitening: 0xD4050FEC
     Initial: L=0x0FEC  R=0xD405
Round  1:    L=0xD405  R=0xA7E7   (k=0x0000  add=0xD405  F=0xA80B)
Round  2:    L=0xA7E7  R=0x9BCA   (k=0x0000  add=0xA7E7  F=0x4FCF)
Round  3:    L=0x9BCA  R=0x9072   (k=0x0000  add=0x9BCA  F=0x3795)
Round  4:    L=0x9072  R=0xBB2F   (k=0x0000  add=0x9072  F=0x20E5)
Round  5:    L=0xBB2F  R=0xE8D7   (k=0x0123  add=0xBC52  F=0x78A5)
Round  6:    L=0xE8D7  R=0xE753   (k=0x4567  add=0x2E3E  F=0x5C7C)
Round  7:    L=0xE753  R=0x092B   (k=0x89AB  add=0x70FE  F=0xE1FC)
Round  8:    L=0x092B  R=0x4966   (k=0xCDEF  add=0xD71A  F=0xAE35)
After final whitening: 0x48454C4C

Recovered: 'HELLO'
```

Figure 5: Decryption Process of Test 1

4.2 TEST 2: Empty message handling (edge case with padding only)

- This Test verifies the correct functioning for a case where there is no data but only padding.
- **Plaintext: ““**
- **Key:** 0123456789ABCDEF

Table 2: Table of Test 2

Test Objectives	**Check empty text for correct processing.**
Test Input	➢ Plaintext: **""** ➢ Key: 0123456789ABCDEF
Action Performed	Encrypt the empty plaintext and obtain ciphertext. Then decrypt with same key.
Expected Result	Encryption process empty plaintext and decryption process gives empty plaintext after using same master key.
Actual Result	➢ Ciphertext: 3F7F5AAF ➢ The Plaintext is empty.
Conclusion	The Test was Successful. Encryption/Decryption process are working as expected.

```
Continue? (y/n): Y
 CHOOSE E- ENCRYPTION AND D- DECRYPTION: E
Key (16 hex digits): 0123456789ABCDEF
Plaintext:

------------------------------------------------------------------

Plaintext: ''
Padded hex: 04040404
Blocks: 1

→ Plain block trace 0x04040404 (Encrypt)
After initial whitening: 0x05274163
     Initial: L=0x0527  R=0x4163
Round  1:    L=0x4163  R=0x1B83   (k=0xCDEF  add=0x0F52  F=0x1EA4)
Round  2:    L=0x1B83  R=0x0B3E   (k=0x89AB  add=0xA52E  F=0x4A5D)
Round  3:    L=0x0B3E  R=0xBAC9   (k=0x4567  add=0x50A5  F=0xA14A)
Round  4:    L=0xBAC9  R=0x7CE7   (k=0x0123  add=0xBBEC  F=0x77D9)
Round  5:    L=0x7CE7  R=0x4307   (k=0x0000  add=0x7CE7  F=0xF9CE)
Round  6:    L=0x4307  R=0xFAE9   (k=0x0000  add=0x4307  F=0x860E)
Round  7:    L=0xFAE9  R=0xB6D4   (k=0x0000  add=0xFAE9  F=0xF5D3)
Round  8:    L=0xB6D4  R=0x9740   (k=0x0000  add=0xB6D4  F=0x6DA9)
After final whitening: 0x3F7F5AAF

Ciphertext (hex):
3F7F5AAF
```

Figure 6: Encryption Process of Test 2

```
Continue? (y/n): Y
 CHOOSE E- ENCRYPTION AND D- DECRYPTION: D
Key (16 hex digits): 0123456789ABCDEF
Ciphertext (hex): 3F7F5AAF

------------------------------------------------------------------

→ Cipher block trace 0x3F7F5AAF (Decrypt)
After initial whitening: 0xB6D49740
     Initial: L=0x9740  R=0xB6D4
Round  1:    L=0xB6D4  R=0xFAE9   (k=0x0000  add=0xB6D4  F=0x6DA9)
Round  2:    L=0xFAE9  R=0x4307   (k=0x0000  add=0xFAE9  F=0xF5D3)
Round  3:    L=0x4307  R=0x7CE7   (k=0x0000  add=0x4307  F=0x860E)
Round  4:    L=0x7CE7  R=0xBAC9   (k=0x0000  add=0x7CE7  F=0xF9CE)
Round  5:    L=0xBAC9  R=0x0B3E   (k=0x0123  add=0xBBEC  F=0x77D9)
Round  6:    L=0x0B3E  R=0x1B83   (k=0x4567  add=0x50A5  F=0xA14A)
Round  7:    L=0x1B83  R=0x4163   (k=0x89AB  add=0xA52E  F=0x4A5D)
Round  8:    L=0x4163  R=0x0527   (k=0xCDEF  add=0x0F52  F=0x1EA4)
After final whitening: 0x04040404

Recovered: ''
```

Figure 7: Decryption Process of Test 2

4.3 TEST 3: Longer message with multiple blocks

- This test confirms that more than one-block-long input is appropriately processed, including processing of several blocks.
- **Plaintext: "**MY NAME IS BADAL KUMAR YADAV "
- **Key:** FEDCBA9876543210

Table 3 : Table of Test 3

Test Objectives	**Verify multi-block encryption/decryption for longer text**
Test Input	➢ Plaintext: "MY NAME IS BADAL KUMAR YADAV" ➢ Key: FEDCBA9876543210
Action Performed	Encrypt long text and decrypt full ciphertext
Expected Result	Recovered plaintext matches original exactly.
Actual Result	➢ Ciphertext: D141FBBBE7DB95130588C4717D22FEEE6ECB3F0A0FE721CA40429D04D4320521 ➢ Recovered: plaintext
Conclusion	The Test was Successful. Multi-block chaining and padding function is correctly arranged.

```
Continue? (y/n): Y
 CHOOSE E- ENCRYPTION AND D- DECRYPTION: E
Key (16 hex digits): FEDCBA9876543210
Plaintext: MY NAME IS BADAL KUMAR YADAV

--------------------------------------------------------------------------------

Plaintext: 'MY NAME IS BADAL KUMAR YADAV'
Padded hex: 4D59204E414D4520495320424144414C204B554D4152205941444156040404
Blocks: 8

→ Plain block trace 0x4D59204E (Encrypt)
After initial whitening: 0xB3859AD6
     Initial: L=0xB385  R=0x9AD6
Round  1:    L=0x9AD6  R=0x2A48   (k=0x3210  add=0xCCE6  F=0x99CD)
Round  2:    L=0x2A48  R=0xDBEF   (k=0x7654  add=0xA09C  F=0x4139)
Round  3:    L=0xDBEF  R=0x0747   (k=0xBA98  add=0x9687  F=0x2D0F)
Round  4:    L=0x0747  R=0xD7A9   (k=0xFEDC  add=0x0623  F=0x0C46)
Round  5:    L=0xD7A9  R=0xA814   (k=0x0000  add=0xD7A9  F=0xAF53)
Round  6:    L=0xA814  R=0x8780   (k=0x0000  add=0xA814  F=0x5029)
Round  7:    L=0x8780  R=0xA715   (k=0x0000  add=0x8780  F=0x0F01)
Round  8:    L=0xA715  R=0xC9AB   (k=0x0000  add=0xA715  F=0x4E2B)
After final whitening: 0xD141FBBB

Ciphertext (hex):
D141FBBBE7DB95130588C4717D22FEEE6ECB3F0A0FE721CA40429D04D4320521
```

Figure 8: Encryption Process of Test 3

```
Continue? (y/n): Y
 CHOOSE E- ENCRYPTION AND D- DECRYPTION: D
Key (16 hex digits): FEDCBA9876543210
Ciphertext (hex): D141FBBBE7DB95130588C4717D22FEEE6ECB3F0A0FE721CA40429D04D43205
21

--------------------------------------------------------------------------------

→ Cipher block trace 0xD141FBBB (Decrypt)
After initial whitening: 0xA715C9AB
     Initial: L=0xC9AB  R=0xA715
Round  1:    L=0xA715  R=0x8780   (k=0x0000  add=0xA715  F=0x4E2B)
Round  2:    L=0x8780  R=0xA814   (k=0x0000  add=0x8780  F=0x0F01)
Round  3:    L=0xA814  R=0xD7A9   (k=0x0000  add=0xA814  F=0x5029)
Round  4:    L=0xD7A9  R=0x0747   (k=0x0000  add=0xD7A9  F=0xAF53)
Round  5:    L=0x0747  R=0xDBEF   (k=0xFEDC  add=0x0623  F=0x0C46)
Round  6:    L=0xDBEF  R=0x2A48   (k=0xBA98  add=0x9687  F=0x2D0F)
Round  7:    L=0x2A48  R=0x9AD6   (k=0x7654  add=0xA09C  F=0x4139)
Round  8:    L=0x9AD6  R=0xB385   (k=0x3210  add=0xCCE6  F=0x99CD)
After final whitening: 0x4D59204E

Recovered: 'MY NAME IS BADAL KUMAR YADAV'
--------------------------------------------------------------------------------
```

Figure 9: Decryption Process of Test 3

4.4 TEST 4: Repetitive input pattern resistance

- This test measures the strength of the cipher against diffusing repetitive inputs, including with a zero key (worst-case scenario).
- **Plaintext:** “AAAAAAAAAAAAAAA “
- **Key:** 0000000000000000

Table 4: Table of Test 4

Test Objectives	**Verify handling of empty plaintext.**
Test Input	➢ Plaintext: " AAAAAAAAAAAAAAA " ➢ Key: 0000000000000000
Action Performed	Encrypt repeated text, then observe ciphertext and finally decrypt the ciphertext.
Expected Result	Ciphertext does not have any apparent repetition; decryption recovers input
Actual Result	➢ Plaintext hex: 41414141414141414141414141414101 ➢ Ciphertext: A5A5E4E4A5A5E4E4A5A5E4E485A5B0A4 ➢ Recovered: "AAAAAAAAAAAAAAAA"
Conclusion	The Test was Successful. Patterns broken by operations; recovery successful

```
Continue? (y/n): Y
 CHOOSE E- ENCRYPTION AND D- DECRYPTION: E
Key (16 hex digits): 0000000000000000
Plaintext: AAAAAAAAAAAAAA

------------------------------------------------------------------

Plaintext: 'AAAAAAAAAAAAAA'
Padded hex: 41414141414141414141414141414101
Blocks: 4

→ Plain block trace 0x41414141 (Encrypt)
After initial whitening: 0x41414141
     Initial: L=0x4141  R=0x4141
Round  1:    L=0x4141  R=0xC3C3   (k=0x0000  add=0x4141  F=0x8282)
Round  2:    L=0xC3C3  R=0xC6C6   (k=0x0000  add=0xC3C3  F=0x8787)
Round  3:    L=0xC6C6  R=0x4E4E   (k=0x0000  add=0xC6C6  F=0x8D8D)
Round  4:    L=0x4E4E  R=0x5A5A   (k=0x0000  add=0x4E4E  F=0x9C9C)
Round  5:    L=0x5A5A  R=0xFAFA   (k=0x0000  add=0x5A5A  F=0xB4B4)
Round  6:    L=0xFAFA  R=0xAFAF   (k=0x0000  add=0xFAFA  F=0xF5F5)
Round  7:    L=0xAFAF  R=0xA5A5   (k=0x0000  add=0xAFAF  F=0x5F5F)
Round  8:    L=0xA5A5  R=0xE4E4   (k=0x0000  add=0xA5A5  F=0x4B4B)
After final whitening: 0xA5A5E4E4

Ciphertext (hex):
A5A5E4E4A5A5E4E4A5A5E4E485A5B0A4
```

Figure 10: Encryption Process of Test 4

```
Continue? (y/n): Y
 CHOOSE E- ENCRYPTION AND D- DECRYPTION: D
Key (16 hex digits): 0000000000000000
Ciphertext (hex): A5A5E4E4A5A5E4E4A5A5E4E485A5B0A4

------------------------------------------------------------------

→ Cipher block trace 0xA5A5E4E4 (Decrypt)
After initial whitening: 0xA5A5E4E4
     Initial: L=0xE4E4  R=0xA5A5
Round  1:    L=0xA5A5  R=0xAFAF   (k=0x0000  add=0xA5A5  F=0x4B4B)
Round  2:    L=0xAFAF  R=0xFAFA   (k=0x0000  add=0xAFAF  F=0x5F5F)
Round  3:    L=0xFAFA  R=0x5A5A   (k=0x0000  add=0xFAFA  F=0xF5F5)
Round  4:    L=0x5A5A  R=0x4E4E   (k=0x0000  add=0x5A5A  F=0xB4B4)
Round  5:    L=0x4E4E  R=0xC6C6   (k=0x0000  add=0x4E4E  F=0x9C9C)
Round  6:    L=0xC6C6  R=0xC3C3   (k=0x0000  add=0xC6C6  F=0x8D8D)
Round  7:    L=0xC3C3  R=0x4141   (k=0x0000  add=0xC3C3  F=0x8787)
Round  8:    L=0x4141  R=0x4141   (k=0x0000  add=0x4141  F=0x8282)
After final whitening: 0x41414141

Recovered: 'AAAAAAAAAAAAAA'
------------------------------------------------------------------
```

Figure 11: Decryption Process of Test 4

4.5 TEST 5: Avalanche effect demonstration

- This test compares two similar texts to analyze how small variations impact the cipher.
- **Plaintext 1:** Test1234
- **Key:** 1122334455667788
- **Plaintext 2:** Test1235
- Same key as plaintext 1

Table 5: Table of Test 5

Test Objectives	**Show strong ciphertext difference from minor plaintext change**
Test Input	➢ A: "Test1234" Key: 1122334455667788 ➢ B: "Test1235" same key
Action Performed	Encrypt both; compare ciphertexts (bit differences)
Expected Result	Ciphertexts differ in many bits (~50% ideal for diffusion)
Actual Result	➢ Ciphertext A: DE175C5CB4401F1E82E72F02 ➢ Ciphertext B: DE175C5CB3C0133782E72F02 ➢ Bit difference: 9.375% (9 bits out of 96)
Conclusion	The Test was Successful. Observable avalanche effect; small change causes noticeable differences.

```
Continue? (y/n): Y
 CHOOSE E- ENCRYPTION AND D- DECRYPTION: E
Key (16 hex digits): 1122334455667788
Plaintext: TEST1234

--------------------------------------------------------------------------------

Plaintext: 'TEST1234'
Padded hex: 54455354313233404040404
Blocks: 3

→ Plain block trace 0x54455354 (Encrypt)
After initial whitening: 0x45676010
     Initial: L=0x4567  R=0x6010
Round  1:    L=0x6010  R=0xEA56   (k=0x7788  add=0xD798  F=0xAF31)
Round  2:    L=0xEA56  R=0x1F68   (k=0x5566  add=0x3FBC  F=0x7F78)
Round  3:    L=0x1F68  R=0x4F0E   (k=0x3344  add=0x52AC  F=0xA558)
Round  4:    L=0x4F0E  R=0xDF08   (k=0x1122  add=0x6030  F=0xC060)
Round  5:    L=0xDF08  R=0xF11F   (k=0x0000  add=0xDF08  F=0xBE11)
Round  6:    L=0xF11F  R=0x3D37   (k=0x0000  add=0xF11F  F=0xE23F)
Round  7:    L=0x3D37  R=0x8B71   (k=0x0000  add=0x3D37  F=0x7A6E)
Round  8:    L=0x8B71  R=0x2BD4   (k=0x0000  add=0x8B71  F=0x16E3)
After final whitening: 0xDE175C5C

Ciphertext (hex):
DE175C5CB4401F1E82E72F02
```

Figure 12: Encryption Process Of plaintext 1

```
Continue? (y/n): Y
 CHOOSE E- ENCRYPTION AND D- DECRYPTION: E
Key (16 hex digits): 1122334455667788
Plaintext: TEST1235

--------------------------------------------------------------------------------

Plaintext: 'TEST1235'
Padded hex: 54455354313233504040404
Blocks: 3

→ Plain block trace 0x54455354 (Encrypt)
After initial whitening: 0x45676010
     Initial: L=0x4567  R=0x6010
Round  1:    L=0x6010  R=0xEA56   (k=0x7788  add=0xD798  F=0xAF31)
Round  2:    L=0xEA56  R=0x1F68   (k=0x5566  add=0x3FBC  F=0x7F78)
Round  3:    L=0x1F68  R=0x4F0E   (k=0x3344  add=0x52AC  F=0xA558)
Round  4:    L=0x4F0E  R=0xDF08   (k=0x1122  add=0x6030  F=0xC060)
Round  5:    L=0xDF08  R=0xF11F   (k=0x0000  add=0xDF08  F=0xBE11)
Round  6:    L=0xF11F  R=0x3D37   (k=0x0000  add=0xF11F  F=0xE23F)
Round  7:    L=0x3D37  R=0x8B71   (k=0x0000  add=0x3D37  F=0x7A6E)
Round  8:    L=0x8B71  R=0x2BD4   (k=0x0000  add=0x8B71  F=0x16E3)
After final whitening: 0xDE175C5C

Ciphertext (hex):
DE175C5CB3C0133782E72F02
```

Figure 13: Encryption Process of plaintext 2

5.0 Evaluation

This section discusses good points and weak points of Avalanche Cloud Feistel Cipher (ACFC) we have constructed. We also discuss where this cipher could be used in real life or in studies.

5.1 Strengths of ACFC Algorithm

- Easy to understand Feistel design ACFC preserves the simple and straightforward Feistel method of dividing the data into two halves. All the steps of encryption and decryption are easy to understand and visualize.
- Simple but useful round function in our round function, we combine addition, which causes carry, and 1 bit rotation. Both of these steps are simple but helpful in disseminating the change in the data.
- Key is used in a smart way (whitening) Before and after the rounds, we mix the whole block with parts of the key. This increases reliance on the key and makes it difficult to guess.
- Represents Avalanche Effect Well Although there are only 8 rounds, a small variation in the message or key bits causes many bits of the result to differ. This is a great example for illustrating how a small variation should have a large impact on the result.
- Very light weight and fast It only uses very basic operations like XOR, addition, and rotation. It is very fast even on the smallest hardware devices.
- Same for encrypt and decrypt You basically have the same code for each. Just reverse the round keys. That is one of the neat parts of using a Feistel Cipher.

5.2 Weakness Points of ACFC Algorithms

- Not tested against strong attacks We have not subjected our hash function to recent methods of attacks for example differential and linear cryptanalysis. We cannot, therefore, determine how secure it really is against hackers.
- The very short key and 64-bit key and 32-bit block size of the key is too small. An ordinary computer can quickly try all the possibilities of the key, so it’s not secure for sensitive data.

- Very basic key splitting We simply divided the key into 8 parts without any mixing or hash value. Anyone knowing how to cut that key can easily try to guess other keys connected with it.
- Small blocks can display patterns As each block is only 32 bits, and since we are processing them one by one without chaining, even for lengthy messages containing many patterns, some patterns can be revealed in the generated output.
- Rotation is only 1 bit Rotation is always only 1 bit. We rotate by 1 bit. That is simple, but not as effective as varying rotations for each round.
- Protects against modification of the message ACFC only conceals the message. It cannot determine whether the person altering the ciphertext did so intentionally.

5.3 Where ACFC Can Be Used

ACFC is not strong enough for real important secrets, but it is very good in these situations:

- Teaching of cryptography in class: Perfect for school and college to show how Feistel ciphers work, what avalanche means, and why we need many rounds.
- Student projects and homework Students can easily code it, change it, test it, and learn from it without hard math or big libraries.
- Fast prototyping and experimentation Useful when a person wants to try new ideas with Feistel + addition + whitening really fast.
- Small demos on tiny devices: Can run on Arduino, Raspberry Pi Pico, or other small boards to do real encryption live.
- Practice and fun challenges: useful when performing tasks in coding clubs, CTF games, or workshops, or when basic protection is sufficient in training labs.

5.4 Final Thought

ACFC is a great teaching cipher. It is easy to understand, fast-paced, easy to follow along with, and it illustrates key concepts nicely that involve blending, distributing change, and key usage. It can certainly not substitute the likes of AES, but it can definitely be a great teaching cipher.

6.0 Conclusion

In this research, we were able to successfully design and implement an altogether new simple symmetric cipher technique named **Avalanche Cloud Feistel Cipher** (ACFC). The primary objective was to design an altogether new cipher technique that explains key concepts in cryptography, including the division of data, processing the data along with the key, the concept of diffusion, and the avalanche effect. We have used the standard Feistel cipher to make the process reverse and easy to understand. We have only added a few basic concepts, including modular addition, 1-bit rotation, key whitening, and splitting the key to make the whole process easy to understand. Each step was tested individually for various types of messages, including text messages, empty messages, longer messages, repeated patterns, and slightly varying messages, while successfully encrypting and decrypting the message using the same key.

By testing and tracing, we could see how the data is changing from one round to another. The findings indicated that ACFC performs as expected: it is easy to learn, demonstrates good avalanche properties even after only 8 rounds, and assists in understanding how modern ciphers mix data with keys. Of course, in real-life protection of confidential data, ACFC is inefficient because of its small block size (only 32 bits) combined with small key size (only 64 bits), simple key scheduling, and vulnerability to strong attacks, meaning it should never be used to preserve valuable data in real life; however, in learning, it is valuable.

Overall, this project has demonstrated the whole process of implementing a cryptography tool from idea formation until the last step of explaining its strength and weaknesses. ACFC can be considered an excellent example for a teaching project. The project allows students to understand better how a block cipher works and why more than one round is important for a mixing process. In the future, this project can act as a base for more advanced projects conducted by students as well. A small but valuable addition has been made to how cryptography could be learned by all.

7.0 References

NIST , 1999. *Data Encryption Standard (DES).* [Online] Available at: https://csrc.nist.gov/files/pubs/fips/46-3/final/docs/fips46-3.pdf

Adomey, M. K. G., 2026. *Introduction to Cryptography.* [Online] Available at: https://www.itu.int/en/ITU-D/Cybersecurity/Documents/01-Introduction%20to%20Cryptography.pdf [Accessed 2026].

Bhatia, K. & Som, S., 2016. *Study on white-box cryptography: Key whitening and entropy attacks,* NOIDA , INDIA: IEEE.

Bing sun, j. l. c. L., 2021. New Approach towards Generalizing Feistel Networks and Its Provable Security. *naasa*, septemper, p. 26.

Floyd, T. L., 2015. *Digital Fundamentals.* 11th Edition, ed. London: Pearson Education Limited.

FORTINET, 2025. *CIA Triad.* [Online] Available at: https://www.fortinet.com/resources/cyberglossary/cia-triad

GOST, 2025. ScienceDirect Topics. *GOST (block cipher) description*, pp. https://www.sciencedirect.com/topics/computer-science/block-cipher/.

GREEKS OF GREEKS, 2025. *What is CIA Triad?.* [Online] Available at: https://www.geeksforgeeks.org/computer-networks/the-cia-triad-in-cryptography/ [Accessed 2026].

greeksforgeeks, 2025. *Cryptography and its Types.* [Online] Available at: https://www.geeksforgeeks.org/computer-networks/cryptography-and-its-types/

greeksofgreek, 2023. *Block Cipher Design Principles.* [Online] Available at: https://www.geeksforgeeks.org/computer-networks/block-cipher-design-principles/

GREEKSOFGREEKS, 2025. *MODULAR ADDITION.* [Online] Available at: https://www.geeksforgeeks.org/engineering-mathematics/modular-addition/ [Accessed 15 JANAURAY 2026].

hackerone, 2025. *Information Security: Principles, Threats, and Solutions.* [Online] Available at: https://www.hackerone.com/knowledge-center/principles-threats-and-solutions

Khurana, D., n.d. *LECTURE 4 BLOCK CIPHER II.* [Online] Available at: https://courses.grainger.illinois.edu/cs498ac3/fa2020/Files/Lecture_4_Scribe.pdf/

Menezes, A. (. A., 1997. *Handbook of applied cryptography.* 1997 ed. london: Boca Raton : CRC Press.

mrajacse.wordpress, 2019. *block cipher and data encryption standarad.* [Online] Available at: https://mrajacse.wordpress.com/wp-content/uploads/2012/01/chapter-3-block-cipher-des1.pdf

[Accessed 2026].

Nagarajan, A., 2017. *DATA-TRANSFORMATION ALGORITHMS,* tEXAS: Council of.

numberanalytics, 2025. *Feistel Cipher Structure Explained.* [Online] Available at: https://www.numberanalytics.com/blog/feistel-cipher-structure-explained/

Politecnico di Torino, 2025. *An Overview on Cryptanalysis of ARX Ciphers.* [Online] Available at: https://crypto.polito.it/content/download/480/2850/file/document.pdf/

SCIENCEDIRECT, 2021. Efficient key-dependent dynamic S-boxes based on permutated elliptic curves. *INFORMATION SCIENCES*, MAY, p. https://www.sciencedirect.com/science/article/abs/pii/S0020025521000335/.

studocu, 2025. *Modern Symmetric-Key Ciphers: Unit 2 Overview and Key Concepts.* [Online]

Available at: https://www.studocu.com/in/document/jawaharlal-nehru-technological-university-kakinada/cryptography-and-network-security/unit-2-cns/96890398/

8.0 APPENDIX

8.1 Python Code of ACFC cipher

```
"""
Avalanche Cloud Feistel Cipher (ACFC)
=====================================
Block size:  32 bit
Key size:    64 bit
Rounds:      8
Round func:  F(R, K) = ROTL1( (R + K) mod 65536 )
Whitening:   pre & post using upper/lower 32 bit of master key
"""

def rotl1_16(x: int) -> int:
    """Left rotate 16-bit value by 1 position"""
    return ((x << 1) & 0xFFFF) | (x >> 15)


def derive_keys(master_key: int) -> tuple[list[int], int, int]:
    if not (0 <= master_key <= 0xFFFFFFFFFFFFFFFF):
        raise ValueError("Key must be 64-bit value (0 to 2⁶⁴-1)")

    round_keys = [(master_key >> (i * 16)) & 0xFFFF for i in range(8)]

    # Whitening keys: upper 32 bit = pre-encrypt / post-decrypt
    #                 lower 32 bit = post-encrypt / pre-decrypt
    w_pre_encrypt  = (master_key >> 32) & 0xFFFFFFFF
    w_post_encrypt = master_key & 0xFFFFFFFF

    return round_keys, w_pre_encrypt, w_post_encrypt


def feistel_block(block32: int, round_keys: list[int],
```

```
            w_pre: int, w_post: int, encrypt: bool = True) -> int:
    """
    Process one 32-bit block.
    Whitening order differs between encrypt and decrypt.
    """
    rks = round_keys if encrypt else round_keys[::-1]

    # Whitening - order depends on direction
    if encrypt:
        block32 ^= w_pre
    else:
        block32 ^= w_post

    L = (block32 >> 16) & 0xFFFF
    R = block32 & 0xFFFF

    # Standard Feistel trick: swap halves at start of decryption
    if not encrypt:
        L, R = R, L

    for rk in rks:
        temp = (R + rk) & 0xFFFF
        f = rotl1_16(temp)
        new_L = R
        new_R = L ^ f
        L, R = new_L, new_R

    # Undo the swap for decryption
    if not encrypt:
        L, R = R, L

    result = (L << 16) | R
```

```
    # Final whitening - opposite order
    if encrypt:
        result ^= w_post
    else:
        result ^= w_pre

    return result


def format_hex(n: int, digits: int = 8) -> str:
    return f"0x{n:0{digits}X}"


def trace_block(block: int, round_keys: list[int], w_pre: int, w_post: int,
                encrypt: bool, label: str = ""):
    """Detailed trace - only for first block"""
    direction = "Encrypt" if encrypt else "Decrypt"
    print(f"\n→ {label} block trace {format_hex(block)} ({direction})")

    # Apply correct whitening start
    current = block ^ (w_pre if encrypt else w_post)
    print(f"After initial whitening: {format_hex(current)}")

    L = (current >> 16) & 0xFFFF
    R = current & 0xFFFF

    if not encrypt:
        L, R = R, L

    print(f"     Initial: L={format_hex(L,4)}  R={format_hex(R,4)}")

    rks = round_keys if encrypt else round_keys[::-1]
```

```
    for i, rk in enumerate(rks, 1):
        temp = (R + rk) & 0xFFFF
        f = rotl1_16(temp)
        new_L = R
        new_R = L ^ f
        print(f"Round {i:2d}:    L={format_hex(new_L,4)}  R={format_hex(new_R,4)}   "
              f"(k={format_hex(rk,4)}  add={format_hex(temp,4)}  F={format_hex(f,4)})")
        L, R = new_L, new_R

    if not encrypt:
        L, R = R, L

    final = (L << 16) | R
    final ^= (w_post if encrypt else w_pre)
    print(f"After final whitening: {format_hex(final)}\n")


# ——— Padding
# ————————————————————————————————————————————————————————————

def pad_pkcs7(data: bytes, bs: int = 4) -> bytes:
    pad = bs - len(data) % bs
    return data + bytes([pad] * pad)


def unpad_pkcs7(data: bytes) -> bytes:
    if not data:
        return b''
    p = data[-1]
    if p < 1 or p > len(data) or data[-p:] != bytes([p] * p):
        raise ValueError("Invalid padding")
    return data[:-p]
```

```
# ——— Text Encrypt / Decrypt ————————————————————————————————

def encrypt_text(text: str, key_hex: str) -> str:
    key = int(key_hex, 16)
    rks, wpre, wpost = derive_keys(key)

    data = text.encode('utf-8')
    padded = pad_pkcs7(data)

    print(f"\nPlaintext: {text!r}")
    print(f"Padded hex: {padded.hex().upper()}")
    print(f"Blocks: {len(padded)//4}")

    blocks_out = []
    trace_done = False

    for i in range(0, len(padded), 4):
        blk = int.from_bytes(padded[i:i+4], "big")
        ct = feistel_block(blk, rks, wpre, wpost, True)

        if not trace_done:
            trace_block(blk, rks, wpre, wpost, True, "Plain")
            trace_done = True

        blocks_out.append(ct)

    return b''.join(b.to_bytes(4, "big") for b in blocks_out).hex().upper()


def decrypt_text(cipher_hex: str, key_hex: str) -> str:
    key = int(key_hex, 16)
    cipher_bytes = bytes.fromhex(cipher_hex)
```

```
    if len(cipher_bytes) % 4 != 0:
        raise ValueError("Ciphertext length must be multiple of 4 bytes")

    rks, wpre, wpost = derive_keys(key)

    blocks_out = []
    trace_done = False

    for i in range(0, len(cipher_bytes), 4):
        blk = int.from_bytes(cipher_bytes[i:i+4], "big")
        pt = feistel_block(blk, rks, wpre, wpost, False)

        if not trace_done:
            trace_block(blk, rks, wpre, wpost, False, "Cipher")
            trace_done = True

        blocks_out.append(pt)

    padded = b''.join(b.to_bytes(4, "big") for b in blocks_out)
    return unpad_pkcs7(padded).decode('utf-8')


# ——— Interactive Menu
# ------------------------------------------------------------

def main():
    print("=== Avalanche Cloud Feistel Cipher (ACFC)  ===\n")
    print("Key: 16 hex digits (64 bits)\n")

    while True:
        try:
```

```
    mode = input(" CHOOSE E- ENCRYPTION AND D- DECRYPTION: ").strip().lower()
    if mode not in ('e','d'):
        print("Only e or d allowed\n")
        continue

    key_str = input("Key (16 hex digits): ").strip().replace(" ", "").replace("0x","")
    if len(key_str) != 16 or not all(c in "0123456789abcdefABCDEF" for c in key_str):
        print("Invalid key – exactly 16 hex digits required\n")
        continue

    if mode == 'e':
        pt = input("Plaintext: ")
        print("\n" + "—"*65)
        ct_hex = encrypt_text(pt, key_str)
        print("Ciphertext (hex):")
        print(ct_hex)
        print("—"*65 + "\n")
    else:
        ct_hex = input("Ciphertext (hex): ").strip().replace(" ", "").replace("0x","")
        if len(ct_hex) % 8 != 0:
            print("Ciphertext hex length must be multiple of 8\n")
            continue
        print("\n" + "—"*65)
        try:
            recovered = decrypt_text(ct_hex, key_str)
            print(f"Recovered: {recovered!r}")
        except ValueError as e:
            print(f"Failure: {e}")
        print("—"*65 + "\n")

    if input("Continue? (y/n): ").strip().lower() != 'y':
```

```
            print("Finished.\n")
            break

        except KeyboardInterrupt:
            print("\nInterrupted.\n")
            break
        except Exception as e:
            print(f"Error: {e}\n")


if __name__ == "__main__":
    main()
```

Click here to navigate on the top

8.2 Encryption process Example:

ACFC Encryption and Decryption Process Example

Input Values

Plaintext: "Cryptography"

Key (64-bit hex): 1234567890ABCDEF

Step 0 – Preparation

1. Convert plaintext to UTF-8 bytes

"Cryptography" → 43 72 79 70 74 6F 67 72 61 70 68 79

2. Apply PKCS#7 Padding (Block size = 4 bytes)

Length = 12 bytes → pad with 4 bytes of value 0x04

Padded bytes:

43 72 79 70 74 6F 67 72 61 70 68 79 04 04 04 04

3. Split into 32-bit blocks (big-endian)

Block 1: 0x43727970 ("Cryp")

Block 2: 0x746F6772 ("togr")

Block 3: 0x61706879 ("aphy")

Block 4: 0x04040404 (padding)

We will show the full 8-round process for Block 1 only. Other blocks follow the same rules.

Key Schedule (performed once)

Master Key: 0x1234567890ABCDEF

Round keys (16-bit each):

K1 = 0xCDEF

K2 = 0x90AB

K3 = 0x5678

K4 = 0x1234

K5 = 0x0000

K6 = 0x0000

K7 = 0x0000

K8 = 0x0000

Whitening keys:

W_pre = 0x12345678 (upper 32 bits)

W_post = 0x90ABCDEF (lower 32 bits)

Encryption Process for Block 1: 0x43727970

Step 1 – Pre-Whitening

block = 0x43727970 XOR 0x12345678 = 0x51462F08

Step 2 – Split

L = 0x5146

R = 0x2F08

Step 3 – Rounds 1 to 8

Round 1 (K1 = 0xCDEF)

temp = (0x2F08 + 0xCDEF) & 0xFFFF = 0xFCF7

F = ROTL1(0xFCF7) = 0xF9EE

new_L = 0x2F08

new_R = 0x5146 XOR 0xF9EE = 0xA8A8

L = 0x2F08, R = 0xA8A8

Round 2 (K2 = 0x90AB)

temp = (0xA8A8 + 0x90AB) & 0xFFFF = 0x3953

F = ROTL1(0x3953) = 0x72A6

new_L = 0xA8A8

new_R = 0x2F08 XOR 0x72A6 = 0x5DAE

L = 0xA8A8, R = 0x5DAE

Round 3 (K3 = 0x5678)

temp = (0x5DAE + 0x5678) & 0xFFFF = 0xB426

F = ROTL1(0xB426) = 0x684C

new_L = 0x5DAE

new_R = 0xA8A8 XOR 0x684C = 0xC0E4

L = 0x5DAE, R = 0xC0E4

Round 4 (K4 = 0x1234)

temp = (0xC0E4 + 0x1234) & 0xFFFF = 0xD318

F = ROTL1(0xD318) = 0xA630

new_L = 0xC0E4

new_R = 0x5DAE XOR 0xA630 = 0xFB9E

L = 0xC0E4, R = 0xFB9E

Round 5 (K5 = 0x0000)

temp = (0xFB9E + 0x0000) & 0xFFFF = 0xFB9E

F = ROTL1(0xFB9E) = 0xF73C

new_L = 0xFB9E

new_R = 0xC0E4 XOR 0xF73C = 0x37D8

L = 0xFB9E, R = 0x37D8

Round 6 (K6 = 0x0000)

temp = (0x37D8 + 0x0000) & 0xFFFF = 0x37D8

F = ROTL1(0x37D8) = 0x6FB0

new_L = 0x37D8

new_R = 0xFB9E XOR 0x6FB0 = 0x942E

L = 0x37D8, R = 0x942E

Round 7 (K7 = 0x0000)

temp = (0x942E + 0x0000) & 0xFFFF = 0x942E

F = ROTL1(0x942E) = 0x285C

new_L = 0x942E

new_R = 0x37D8 XOR 0x285C = 0x1F84

L = 0x942E, R = 0x1F84

Round 8 (K8 = 0x0000)

temp = (0x1F84 + 0x0000) & 0xFFFF = 0x1F84

F = ROTL1(0x1F84) = 0x3F08

new_L = 0x1F84

new_R = 0x942E XOR 0x3F08 = 0xAB26

L = 0x1F84, R = 0xAB26

Step 4 – Combine Halves

block = (0x1F84 << 16) | 0xAB26 = 0x1F84AB26

Step 5 – Post-Whitening

ciphertext = 0x1F84AB26 XOR 0x90ABCDEF = 0x8F2F66E9

Final Result for Block 1:

Plaintext block: 0x43727970 ("Cryp")

Ciphertext block: 0x8F2F66E9

Other blocks follow the same encryption procedure.

Click here to navigate up

8.3 Decryption Process Example

Step 1 – Remove Post-Whitening

block = ciphertext XOR W_post = 0x8F2F66E9 XOR 0x90ABCDEF = 0x1F84AB26

Step 2 – Split

L = 0x1F84

R = 0xAB26

Step 3 – Rounds 8 down to 1 (reverse order)

Round 8 (K8 = 0x0000)

temp = (R + K8) & 0xFFFF = 0xAB26

F = ROTL1(0xAB26) = 0x1564

old_L = R XOR F = 0xAB26 XOR 0x1564 = 0xB852

old_R = L = 0x1F84

L = 0xB852, R = 0x1F84

Round 7 (K7 = 0x0000)

temp = (R + K7) & 0xFFFF = 0x1F84

F = ROTL1(0x1F84) = 0x3F08

old_L = R XOR F = 0x1F84 XOR 0x3F08 = 0x204C

old_R = L = 0xB852

L = 0x204C, R = 0xB852

(…continue rounds 6 → 1 similarly…)

Step 4 – Combine Halves

block = (L << 16) | R

Step 5 – Remove Pre-Whitening

plaintext = block XOR W_pre = 0x43727970

Final Result:

Recovered plaintext block: 0x43727970 ("Cryp")

All blocks together reconstruct the original plaintext "Cryptography".

Click here to navigate up